

Client-Side Redaction of Swedish Personally Identifiable Information

maskera: deterministic rules and a 40 MB on-device NER model
that mask PII before text reaches an LLM, a log, or analytics

Joel Hägvall

`maskera.dev` `github.com/joelhagvall/maskera`

July 5, 2026 · Whitepaper v1.3

Abstract

maskera is an open-source (MIT) library that redacts Swedish personally identifiable information locally, in the browser or in the caller's own server process, before text is sent anywhere. Structured identifiers (personnummer, organisation numbers, phone numbers, bank account references, and others) are caught by deterministic, checksum-validated rules. Names, places, organisations, and street addresses in free text are caught by a purpose-built Swedish NER model of 40 MB, small enough to run where the text already is, including inside a browser tab. The input text never leaves the caller's environment, no telemetry is collected, and the mapping that restores masked values is held locally by the caller. Measured on a small independent Swedish gold set, the pipeline flagged 98% of PII entities and achieved the highest typed F_1 among the public Swedish NER models tested, at less than a tenth of their size. maskera is a data-minimisation and data-protection-by-design tool, not a compliance guarantee; this document reports method and limitations with the same candour as the results. Intended readers: data protection officers, security teams, architects, and legal reviewers.

1 The problem: free text leaks PII

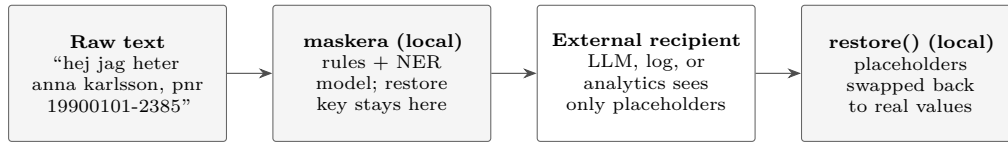
Swedish organisations now send user-generated text to more external systems than ever: large language models behind third-party APIs (frequently hosted outside the EU/EEA), log platforms, analytics pipelines, and support tooling. That text is largely free-form, and free-form text carries personal data: customers type their personnummer into chat windows, case workers mention names and home addresses in ticket notes, patients describe their situation with full identities attached.

Every such flow is a processing operation under data-protection law. Where the recipient is a third-party vendor, questions of processing agreements and third-country transfers follow, and every system that stores the raw text widens the incident surface. The simplest way to reduce all of these risks at once is to not send the personal data at all.

The established technical countermeasures share two weaknesses. Server-side DLP services and redaction proxies only see the text *after* it has left the client, and they are themselves one more system processing raw personal data. The publicly available Swedish NER models, for their part, are on the order of 500 MB; they require a dedicated inference service with a GPU or a strong CPU, which in practice again means a new system that sees the raw text.

maskera takes the opposite approach: redact the text *where it already is*, in the browser before a form is submitted, or inside the caller's own server process before the outbound API call. No new data recipient is introduced.

2 Architecture: two layers, rules first



The redaction flow is: raw text $\rightarrow$ `redact()` $\rightarrow$ masked text with placeholders $\rightarrow$ LLM / log / analytics $\rightarrow$ `restore()` locally. The external recipient only ever sees placeholders such as `[PERSON_1]` and `[PERSONNUMBER_1]`; shaded boxes run inside the caller’s environment.

2.1 Layer 1: deterministic rules (`@maskera/core`)

Structured Swedish identifiers are matched by patterns that are checksum-validated wherever a checksum exists: personnummer, samordningsnummer, organisation numbers, phone numbers, e-mail addresses, postcodes, bankgiro, plusgiro, IBAN, card numbers, IP addresses, and URLs. Validation is what makes the layer precise; “2019-2024” is not read as a bankgiro number, because its checksum fails. The layer is deterministic, synchronous, and has zero dependencies, and can be used entirely on its own, for example for log masking.

2.2 Layer 2: a free-text NER model (`maskera`)

What rules can never catch, such as free-text names written without capital letters, is caught by `maskera-sv-ner`, a purpose-built Swedish named-entity-recognition model with four entity types: person, location, organisation, and street address. It handles text the way people actually type it: all lowercase (“hej jag heter anna karlsson”), ALL CAPS, and genitive forms. The model is 40 MB and runs in the browser (WebGPU or WASM) and in Node.

2.3 The hybrid rule: the deterministic layer always wins

When both layers run, any model detection that overlaps a rule detection is dropped. A personnummer is therefore always classified by checksum logic, never by a probabilistic guess. Within one call, the same value always receives the same placeholder, so a language model can reason about `[PERSON_1]` consistently without ever seeing the underlying value. Restoration happens locally: the placeholder-to-value mapping never leaves the caller’s environment, and must never be logged or stored next to the redacted text.

3 Privacy model: what leaves the device

The caller’s input text never leaves the caller’s environment. All redaction runs locally, in the browser or in the caller’s Node process. The packages contain no telemetry and phone home to nothing. The claim is verifiable: the code is open, and the browser’s developer tools show that typed text triggers no network requests.

For completeness, two network calls exist in the default setup, both fetching static assets and never content:

- **Model weights** (~40 MB), downloaded once from the Hugging Face Hub or from a host of the caller’s choosing, then cached.
- **The WASM runtime** for Transformers.js, loaded from a CDN.

Both can be eliminated. The model is a set of static files that can be served from the caller’s own origin, and `allowRemoteModels: false` forbids remote fetches outright. In that configuration maskera makes zero external requests, which suits procurement requirements that no new subprocessors be introduced, as well as air-gapped environments. The rule layer has no network capability at all.

Mode	Network requests	New subprocessors	Best for
Default	static assets once (HF Hub, CDN)	possibly HF/CDN	evaluation, demos
Self-hosted model	caller’s own origin only	none	production
Rules only	none	none	logs, pipelines
Air-gapped	none	none	strict environments

In every mode, including the default, the requests carry no user content: the only things ever fetched are the static model and runtime files.

4 Model provenance and reproducibility

In a vendor assessment of AI components, two questions dominate: what was the model trained on, and can the answer be checked? For `maskera-sv-ner`, both answers are public.

- **Base model:** `KBLab/bert-base-swedish-cased`, the National Library of Sweden’s BERT, released under CC0. It was fine-tuned for entity recognition, then distilled into a smaller student model quantised to a 40 MB ONNX artifact.
- **Training data:** approximately 24,000 synthetic, template-generated Swedish sentences (the generator is public in the repository and contains no real personal data), plus approximately 6,900 real news sentences from the Swedish NER Corpus, a public, openly licensed research dataset of already-published text about public figures.
- **No user data:** nothing was scraped or collected for this project, and no data from people who use maskera enters training (no such collection exists).
- **Reproducibility:** the entire chain, from data generation through training, distillation, ONNX export, and evaluation, exists as runnable scripts in the repository. An outside auditor can follow and re-create every step.

This openness matters both for the caller’s own GDPR assessment of the component and for the transparency and documentation duties that follow from the EU AI Act.

5 Measured quality

All numbers below were measured on **2026-07-04** against the published artifact (`maskera@0.4.1`, q4 quantisation) using **exact character spans**, the strict criterion under which both start and end offsets must match. The canonical, always-current document is `docs/BENCHMARKS.md` in the repository; this section is a dated snapshot, and where the two disagree, `BENCHMARKS.md` wins. In a privacy setting the number to weigh most heavily is **leakage**: the share of entities missed entirely and thus left unmasked.

5.1 Curated corpus (upper bound, regression tracker)

148 hand-authored Swedish sentences, 204 free-text entities, including hard cases (lowercase, ALL CAPS, genitives, deliberate traps). The corpus shares an author with the training-data generator and should be read as an upper bound, not a universal score.

Metric	Result	Meaning
Precision	95.2%	of the model’s detections, how many were correct
Recall	97.5%	of the real entities, how many were found
Span F_1	96.4%	harmonic mean, label-agnostic
Leakage	1.0%	entities missed entirely, 2 of 204

5.2 Independent gold corpus (honest floor)

22 verbatim sentences of Swedish Wikipedia prose, 58 entities, written by others and held out of all training data. A small sample; read it as a directional independent floor.

Metric	Result	Meaning
Precision	90.0%	of the model’s detections, how many were correct
Recall	93.1%	of the real entities, how many were found
Span F_1	91.5%	harmonic mean, label-agnostic
Leakage	1.7%	entities missed entirely, 1 of 58

5.3 Comparison with public Swedish NER models

Same independent gold corpus, same harness, overlap matching, labels mapped to person, location, and organisation. “Flagged” is the share of entities marked at all, under any label, i.e. the safety number.

Model	Size	Flagged	Typed F_1
maskera (runs in the browser)	40 MB	0.98	0.96
KBLab lowermix reallysimple-ner	~475 MB	1.00	0.94
NB AI-Lab scandi-ner	~500 MB	1.00	0.94
KB-NER (the SUC classic)	~475 MB	1.00	0.92
KBLab reallysimple-ner	~475 MB	0.98	0.91
RecordedFuture Swedish-NER	~500 MB	1.00	0.88

The full tables, all-lowercase results, method notes, and known exceptions are in `BENCHMARKS.md`. One known exception is reported openly: on text entirely without capital letters, KBLab’s lowermix model leads (typed F_1 0.90 versus maskera’s 0.86). None of the alternative models detects street addresses or fits in a web application.

5.4 Latency

Measured **2026-07-05** on an Apple M4 Pro laptop against the curated corpus (average sentence 46 characters); warm figures are per sentence, median and 95th percentile. The browser rows run the production demo bundle with the self-hosted model, exactly what maskera.dev ships. A real first visit adds the network download of the ~40 MB model; returning visitors read it from the browser cache.

Environment	Cold start	Warm (median / p95)	Notes
Browser, WASM (demo default)	0.19 s	50 / 58 ms	no GPU needed
Browser, WebGPU	0.21 s	3.7 / 4.6 ms	opt-in, slower first run
Node (native CPU runtime)	0.21 s	3.5 / 5.1 ms	server-side
Rules only (@maskera/core)	none	~0.002 ms	no model, no network

In practice: masking a chat message costs tens of milliseconds in the browser and single-digit milliseconds on a server, and the deterministic rule layer is effectively free. The measurement scripts live in the repository, under `packages/ner/eval/` and `apps/demo/scripts/`.

5.5 Continuous verification

Quality is not a one-off measurement. Every code change in the project re-runs the evaluation against the published model with an F_1 floor (0.90) and a leakage ceiling (0.08) that fail the build on regression, and a weekly automated canary re-measures the published model against the latest runtime version and raises an alarm on drift. Both pipelines are public in the repository’s CI.

6 Regulatory positioning

This section describes how maskera relates to the regulatory frameworks; it is not legal advice.

- **Data minimisation (Art. 5(1)(c) GDPR).** maskera’s function is that less personal data reaches third parties: what is masked before the API call is never processed by the recipient.
- **Pseudonymisation (Art. 4(5) GDPR).** Masked text with a locally held restore key is pseudonymisation: the data can no longer be attributed to a person without the additional information the caller keeps separately. Pseudonymised text may still constitute personal data, but the measure is explicitly recognised as a safeguard in Art. 25 (data protection by design) and Art. 32 (security of processing).
- **Third-country transfers (Chapter V GDPR).** Personal data that is never sent to a vendor outside the EU/EEA needs no transfer mechanism. Masking before the call shrinks the transfer question accordingly; it does not replace the caller’s analysis of whatever is still sent.
- **The EU AI Act.** maskera supports the data-protection objectives and is fully documented and reproducible, which eases the caller’s vendor and system documentation. The classification of the caller’s AI system is not changed by maskera itself.

maskera is data minimisation, not a compliance guarantee. Compliance is decided by the caller’s whole system: legal basis, contracts, storage, access, and retention. The caller remains the data controller, and maskera makes no legal claims.

7 Limitations

A privacy tool that overstates its accuracy is more dangerous than no tool at all, so the limitations are reported as plainly as the results.

- **The model misses things.** The rule layer is deterministic and highly reliable for structured identifiers, but free-text recognition is probabilistic. Unusual names, heavy misspellings, and OCR noise will sometimes slip through. On independent text the leakage is low (1 of 58 entities) but not zero.
- **Four entity types, one language.** The model recognises persons, locations, organisations, and street addresses in Swedish text. Data outside those types, and PII in other languages, is not caught by the model (structured identifiers are caught by the rules regardless).
- **The independent test set is small.** 22 sentences give a direction, not a final grade. A larger independent gold set is the project’s top measurement priority, and no annotated test set yet exists for the target domains of support, healthcare, and legal text.
- **Defence in depth.** maskera is a strong first layer, not the only one. Keep access controls, processing agreements, logging policies, and human review in high-stakes flows.

8 Threat model: what maskera does not protect against

Data minimisation has a boundary, and a reviewer should know where it runs. maskera does not protect against:

- **Sensitive content that is not an entity.** maskera masks identifiers and named entities. A sentence can remain deeply revealing without containing either: a health condition, a workplace conflict, or a situation described precisely enough to point at one person all survive redaction intact. Masking makes text less identifying; it does not make it harmless.
- **Mishandling of the restore mapping.** The placeholder-to-value mapping is the key that undoes the pseudonymisation. If the caller logs it, stores it next to the redacted text,

or sends it downstream, the protection is reversed outside maskera’s control. Keep the mapping in memory, short-lived, and local.

- **Whatever happens after redaction.** maskera controls what the recipient receives, not what the caller’s downstream logic or the recipient does with it. An application that re-attaches identifiers after restoration and then logs the result has left the protected path.
- **Re-identification from context.** As noted under regulatory positioning, redacted text may still constitute personal data. Rare combinations of unmasked facts (a job title, a small town, a date) can identify a person even with every entity masked.

None of this is unusual; it is the standard boundary of any redaction layer. It is stated here so that the caller’s remaining controls, described in the next section, are chosen deliberately rather than assumed away.

9 Recommended deployment

For production use, the project’s operations guide (`docs/PRODUCTION.md`) recommends:

- Redact *before* the text reaches the language model, the log, or the analytics pipeline, never after.
- If the model layer fails, fall back to the rule layer, never to raw text.
- Never log the restore key and never store it next to the redacted text; restoration happens only inside the caller’s environment.
- Self-host the model files if the Hugging Face Hub is not an acceptable dependency; zero external requests are then made.
- Evaluate against your own text: build a small domain corpus of real (internal, consented) sentences and run it in your CI. The metric that matters is leakage. Sentences the model misses in production are added to the corpus, so every future model update is graded against your own failures.
- Human review remains in place for high-stakes flows (legal, healthcare, financial decisions).

9.1 Reviewer checklist

The same recommendations in checklist form, for pasting into an internal security or privacy review:

- ☐ Redaction runs *before* the LLM call, the log write, and the analytics event.
- ☐ The restore mapping is never logged and never stored next to the redacted text.
- ☐ Model files are self-hosted (or `allowRemoteModels: false` is set); zero external requests confirmed in the network tab.
- ☐ On model failure the system fails closed to the rule layer, never open to raw text.
- ☐ A domain eval corpus of real (consented) sentences runs in CI, gated on leakage.
- ☐ Human review remains in place for high-stakes flows.

10 Auditing and verification

Everything claimed in this document can be checked without asking us:

- **The code:** MIT-licensed, open in full at <https://github.com/joelhagvall/maskera>.
- **The model:** published openly on Hugging Face (`joelhagvall/maskera-sv-ner`) with a model card. The shipped artifact is `onnx/model_q4.onnx`, SHA-256

f34aa2d9e2272aa8cadded3ff819827d858cb5ab07c9b40f2bc4d1d147dd5625;
the current hash always lives in docs/BENCHMARKS.md.

- **The numbers:** canonical in docs/BENCHMARKS.md, dated, and reproducible with documented commands against the published model.
- **The training:** the whole pipeline exists as runnable scripts in training/.
- **The network claims:** verifiable in the browser’s developer tools against the live demo at <https://maskera.dev>.

Appendix: the sentences the model is known to miss

The evaluation harness prints every leak verbatim and nothing is filtered. As of the artifact above, three gold entities across both corpora have no overlapping detection; they are published here deliberately. In each sentence every other entity is caught. On provenance: the two curated sentences are hand-authored test cases with invented names, and the gold-real sentence is verbatim, already-published Wikipedia prose about public figures. No user data and no real private persons appear below.

Input	Missed	Type	Reading
“Klarna rekryterade Daniel från Spotify i fjol.”	Klarna	ORG	sentence-initial org in subject position
“hejhej det är fatima igen, hör av dig när du kan”	fatima	PER	lowercase text interacting with name rarity (see below)
“Den 6 mars 2018 besökte Löfven Vita huset och hade sitt första officiella möte med [...]”	Vita huset	LOC	metonymic building name

The second row deserves a sharper reading than “no casing cues”: capitalised “Fatima” *is* caught, and so is lowercase “anna”. The failure is the combination of lowercase text and a name further from the training distribution, i.e. robustness across name origins in casing-free text. That gap is stated openly and targeted: the next data round adds lowercase augmentation, and the project’s gold-set plan requires invented names spanning many origins, not only Swedish ones.

These are also why the rule layer exists and why leakage, not F_1 , is the gated metric: the failure modes are visible, characterised, and tracked in the public regression corpus.